

SIGNALS

Mohammad Rehan

2520030066

1. Signal.c

```
rehan_lenovo@Rehan-Lenovo x + v
GNU nano 8.7.1 signal.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig){
printf("\nSIGINT recieved!\n");
printf("Signal number = %d\n",sig);
}

int main()
{
signal(SIGINT, handler);

while(1)
{
printf("Program is running...\n");
sleep(2);
}
return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo:~/OS$ nano signal.c
rehan_lenovo@Rehan-Lenovo:~/OS$ gcc signal.c -o signal
rehan_lenovo@Rehan-Lenovo:~/OS$ ./signal
Program is running...
Program is running...
^C
SIGINT recieved!
Signal number = 2
Program is running...
Program is running...
Program is running...
Program is running...
^XProgram is running...
^Z
[1]+  Stopped                  ./signal
rehan_lenovo@Rehan-Lenovo:~/OS$ |
```

2. Sigterm

```
rehan_lenovo@Rehan-Lenovo x + v
GNU nano 8.7.1 sigterm.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if (sig == SIGTERM)
        printf("\nSIGTERM received!\n");
}

int main()
{
    signal(SIGTERM, handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for SIGTERM...\n");

    while (1)
    {
        sleep(2);
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo:~/OS$ nano sigterm.c
rehan_lenovo@Rehan-Lenovo:~/OS$ gcc sigterm.c -o sigterm
rehan_lenovo@Rehan-Lenovo:~/OS$ ./sigterm
Process ID: 2804
Waiting for SIGTERM...

SIGTERM received!
^C
rehan_lenovo@Rehan-Lenovo:~/OS$ ./sigterm
Process ID: 2853
Waiting for SIGTERM...
^C
rehan_lenovo@Rehan-Lenovo:~/OS$
```

3. Sigchild

```
rehan_lenovo@Rehan-Lenovo x + v
GNU nano 8.7.1 sigchild.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void child_handler(int sig)
{
    printf("SIGCHLD received: Child has terminated\n");
    wait(NULL);
}

int main()
{
    signal(SIGCHLD, child_handler);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child is running...\n");
        sleep(2);
        printf("Child is exiting...\n");
    }
    else
    {

```

Output:

```
rehan_lenovo@Rehan-Lenovo x + v
rehan_lenovo@Rehan-Lenovo:~/OS$ nano sigchild.c
rehan_lenovo@Rehan-Lenovo:~/OS$ gcc sigchild.c -o sigchild
rehan_lenovo@Rehan-Lenovo:~/OS$ ./sigchild
Parent is waiting...
Child is running...
Child is exiting...
SIGCHLD received: Child has terminated
Parent finished.
rehan_lenovo@Rehan-Lenovo:~/OS$ |
```

4. Sigpipe

```
rehan_lenovo@Rehan-Lenovo x + v
GNU nano 8.7.1 sigpipe.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigpipe(int sig)
{
    printf("\nSIGPIPE received!\n");
    printf("The reading end of the pipe is closed.\n");
    exit(1);
}

int main()
{
    int fd[2];

    signal(SIGPIPE, handle_sigpipe);

    pipe(fd);

    printf("Pipe created.\n");

    close(fd[0]);

    printf("Reading end closed.\n");

    printf("Trying to write to the pipe...\n");
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo x + v
rehan_lenovo@Rehan-Lenovo:~/OS$ nano sigpipe.c
rehan_lenovo@Rehan-Lenovo:~/OS$ gcc sigpipe.c -o sigpipe
rehan_lenovo@Rehan-Lenovo:~/OS$ ./sigpipe
Pipe created.
Reading end closed.
Trying to write to the pipe...

SIGPIPE received!
The reading end of the pipe is closed.
rehan_lenovo@Rehan-Lenovo:~/OS$ |
```

5. Sigsegv

```
rehan_lenovo@Rehan-Lenovo x + v
GNU nano 8.7.1 sigsegv.c *
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigsegv(int sig)
{
    printf("\nSIGSEGV received!\n");
    printf("Invalid memory access detected.\n");
    exit(1);
}

int main()
{
    signal(SIGSEGV, handle_sigsegv);

    printf("Program started.\n");

    int *p = NULL;

    printf("Trying to access invalid memory...\n");

    *p = 10;

    printf("Program completed.\n");
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo x + v
rehan_lenovo@Rehan-Lenovo:~/OS$ nano sigsegv.c
rehan_lenovo@Rehan-Lenovo:~/OS$ gcc sigsegv.c -o sigsegv
rehan_lenovo@Rehan-Lenovo:~/OS$ ./sigsegv
Program started.
Trying to access invalid memory...

SIGSEGV received!
Invalid memory access detected.
rehan_lenovo@Rehan-Lenovo:~/OS$ |
```